Integrationstesten kørt lokalt

rigtig MariaDB i stedet for Docker

6/11 2026. `plugins/tests/integration.sh` har ligget i repoet og kørt i CI, men var **aldrig blevet afviklet her**. Denne fil er opskriften der fik den til at køre i en cloud-worktree uden Docker, og listen over hvad kørslen afslørede. Resultat efter rettelserne og 7.41.1: **249 PASS / 0 FAIL** mod WordPress 7.1.4 med PHP 8.2.32 + MySQL 8.0.36, og smoke-suiten **523 PASS**. Samme script er kørt mod hele CI-matrixen (6.4, 7.1, latest); 6.4 giver 184 PASS, fordi ability-grupperne springes over. Opskriften nedenfor bruger MariaDB; de nyeste matrix-kørsler brugte MySQL 8.0.36, men scriptet og resultaterne er de samme. Se wordpress-matrix-lokalt-2026-11-06.md.

Opdateret 12/11 2026: kørt igen på plugin **7.79.0** med WordPress latest, PHP **8.5.8**, MariaDB **11.4.5** og wp-cli som phar: **267 PASS / 0 FAIL**. Den statiske PHP i `tools/_deps/bin/php` har nu `mysqli`, så bulk-PHP-downloadet er ikke længere nødvendigt; se §6.

1. Hvorfor Docker ikke kunne bruges

Forsøg	Resultat
docker ps	permission denied while trying to connect to the docker API at unix:///var/run/docker.sock — soklen er root:docker, brugeren er kun i gruppen freebuff
sudo	sudo: The "no new privileges" flag is set, which prevents sudo from running as root
mysqld, mariadbd, mysql, php, wp, composer	findes ikke på PATH

`.tools/php` (den statiske binær fra AGENTS §7.3) har `pdo_mysql`, `mysqlnd` og `sqlite3`, men **ikke mysqli** — og WordPress kræver `mysqli`. SQLite-drop-in blev fravalgt, fordi pluginnet bruger MySQL-specifik SQL (`ON DUPLICATE KEY UPDATE ... VALUES(col)`, `IF()`, `SHOW TABLES LIKE`) på netop de stier testen rammer (klikaggregering), så SQLite-fejl ville være artefakter og ikke pluginfejl.

2. Opskriften

To downloads: MariaDBs bintar (349 MB, ~8 MB/s målt) og static-php-cli **bulk** (25,7 MB), som i modsætning til `common` har `mysqli`.

```
cd /tmp
curl -sSL -O
https://archive.mariadb.org/mariadb-11.4.5/bintar-linux-systemd-x86_64/mariadb-11.4.5-linux-systemd
-x86_64.tar.gz
curl -sSL -O https://dl.static-php.dev/static-php-cli/bulk/php-8.3.14-cli-linux-x86_64.tar.gz
mkdir -p mariadb bulkphp
tar xzf mariadb-11.4.5-linux-systemd-x86_64.tar.gz -C mariadb --strip-components=1
tar xzf php-8.3.14-cli-linux-x86_64.tar.gz -C bulkphp
./bulkphp/php -m | grep mysqli            # mysqli, mysqlnd, pdo_mysql, curl, mbstring, zip, gd
```

Databasen initialiseres og startes uden root (datadir og socket i `/tmp`, så intet rører systemet):

```
./mariadb/scripts/mariadb-install-db --no-defaults --basedir=/tmp/mariadb \
    --datadir=/tmp/mariadb-data --auth-root-authentication-method=normal --skip-test-db
setsid nohup ./mariadb/bin/mariadbd --no-defaults --basedir=/tmp/mariadb
--datadir=/tmp/mariadb-data \
    --socket=/tmp/mariadb.sock --port=3306 --bind-address=127.0.0.1 --pid-file=/tmp/mariadb.pid \
    --log-error=/tmp/mariadb.err >/tmp/mariadb.log 2>&1 </dev/null &
./mariadb/bin/mariadb -h 127.0.0.1 -u root -e "ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';
    CREATE USER IF NOT EXISTS 'root'@'127.0.0.1' IDENTIFIED BY 'root';
    GRANT ALL ON *.* TO 'root'@'127.0.0.1' WITH GRANT OPTION; FLUSH PRIVILEGES;"
```

Testen forventer `root`/`root` på `127.0.0.1:3306`, `php` og `wp` på `PATH` samt en MySQL-klient. Fire shims i `/tmp/bin` dækker det (mønsteret kan genbruges til enhver sandkasse):

```
mkdir -p /tmp/bin
printf '#!/bin/sh\nexec /tmp/bulkphp/php -d memory_limit=512M "$@"\n' > /tmp/bin/php
printf '#!/bin/sh\nexec /tmp/bulkphp/php -d memory_limit=512M /tmp/wp-cli.phar "$@"\n' >
/tmp/bin/wp
for n in mariadb mysql mariadb-dump mysqldump mariadb-admin mysqladmin; do
    printf '#!/bin/sh\nexec /tmp/mariadb/bin/%s "$@"\n' "$n" > "/tmp/bin/$n"
done
chmod +x /tmp/bin/*
PATH=/tmp/bin:$PATH bash plugins/tests/integration.sh
```

3. De fem fælder undervejs

1. **wp core download dør på 128 MB.** Den statiske PHP har `memory_limit = 128M`, og wp-clis `Extractor` bruger mere: `Fatal error: Allowed memory size of 134217728 bytes exhausted ... in WP_CLI/Extractor.php`. Må ikke rettes i repoet — shimmen sender `-d memory_limit=512M`.
2. **mariadb skal findes som netop mariadb.** wp-cli leder efter MariaDB-klienten under det navn og fejler ellers med `/usr/bin/env: 'mariadb': No such file or directory`, hvorefter `wp db create` melder «Cannot select database». Derfor shims for både `mariadb`, `mysql` og dump/admin-navnene.
3. **--no-defaults må ikke stå først i en shim.** wp-cli sender sine egne flag først, og klienten afviser da `unknown option '--no-defaults'`. Shimmen uden `--no-defaults` virker, fordi der ikke ligger nogen `my.cnf` i miljøet (`/etc/mysql` findes ikke).
4. **WordPress gætter sin egen URL.** Installeret i `/tmp/wordpress` uden et eksplicit `home`/`siteurl` svarede sitet `http://127.0.0.1:8080/wordpress`, så alle absolutte link-checks (`/go/<id>/`, RSS-feedet) sammenlignede mod den gale sti. Scriptet sætter nu `home` og `siteurl` til `$URL` efter install.
5. **libaio var ikke et problem.** Den mangler i miljøet, men MariaDB's bintar bruger Linux native AIO og startede uden den.

4. Hvad kørslen afslørede

Første gennemløb: 1 fatal og 12 røde checks. Det vigtigste fund var ikke i scriptet:

- **Providerskift efterlod den gamle providers model.** Fordi 7.39.1 gemmer den model «Provider default» resolver til, stod `ai_model` stadig som `claude-opus-5-5` efter et skift til AIMLAPI. AIMLAPI udbyder `anthropic/claude-opus-5.5`, ikke Anthropic-id'et, så hvert første kald fejlede indtil fallback'en havde repareret feltet. `SGAP_Settings::keep_provider_model()` lytter nu på `update_option_sgap_ai_provider` og erstatter en model den nye provider ikke udbyder med dennes standard — og rører intet når kataloget ikke kan læses, så en kortvarigt utilgængelig provider ikke sletter et navn der måske er fint.
- **HTTP-mocken kendte ikke /v1/models.** 7.39.0 indførte katalog-kaldet, og mocken svarede på det GET som om det var en chat-POST (`json_decode(null)` → `array_map(null)` → fatal). Den svarer nu med et lille katalog, og katalog-GET'et logges ikke som et AI-kald, så `$ai[0]` fortsat er chat-requesten.
- **En udgået model kan ikke gemmes — heller ikke af testen.** Testen skrev `claude-made-up` med `update_option()`, som den nye gemme-kontrol nu afviser (det er pointen). En gammel side har værdien i tabellen, så testen skriver den nu direkte og rydder options-cachen; og den nye kontrol prøves separat.
- **Testen kunne ikke køres to gange.** Databaseskemaet og `hello-dolly` fra første kørsel fik duplicate-queue og «already installed» til at fejle. Scriptet dropper databasen, rydder `wp-content/plugins` og sletter `debug.log`, så hvert gennemløb starter ens.
- **FAIL inde i eval-filerne fejlede ikke kørslen.** `integration.php`, `admin-tools.php` og `automation.php` printer deres egne PASS/FAIL-linjer; et `FAIL` var kun en linje i loggen mens scriptet

exit'ede 0. `run_eval()` griber det nu.

- **To forældede forventninger i admin-tools.php:** `update-settings` er med vilje ikke et chat-tool siden 7.38.3, og checken «saving the model re-tests the key» gemte en værdi der allerede stod i feltet — altså ingen ændring at teste.

5. Sådan læses resultatet

```
INTEGRATION EXIT=0   PASS=249 FAIL=0
SMOKE       EXIT=0   PASS=523 FAIL=0
php -l på alle plugin-filer: rent
```

De fire modelpunkter bevises nu **inde i det rigtige WordPress**, ikke kun i smoke: udgået navn markeres «not offered any more», dropdownen er grupperet med standarden øverst og kun udbudte id'er, et afvist navn gensendes på standarden med en sætning om hvad der blev skiftet væk fra, og «Provider default» gemmes som den model der reelt bruges.

6. Kørslen 12/11 2026 — 7.79.0 og de nye sager

Samme opskrift, men med den PHP der allerede ligger i worktreen. `tools/_deps/bin/php` er 8.5.8 og har `mysqli`, `mysqlnd` og `pdo_mysql`, så kun MariaDB-bintaren og wp-cli skulle hentes:

```
# MariaDB 11.4.5 som i §2; wp-cli som phar i stedet for Composer
curl -sSL -o /tmp/wp-cli.phar
https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
mkdir -p /tmp/bin
printf '#!/bin/sh\nexec /workspace/.../tools/_deps/bin/php -d memory_limit=-1 "$@"\n' >
/tmp/bin/php
printf '#!/bin/sh\nexec /workspace/.../tools/_deps/bin/php -d memory_limit=-1 /tmp/wp-cli.phar
"$@"\n' > /tmp/bin/wp
for n in mariadb mysql mariadb-dump mysqldump mariadb-admin mysqladmin; do
    printf '#!/bin/sh\nexec /tmp/mariadb/bin/%s "$@"\n' "$n" > "/tmp/bin/$n"
done
chmod +x /tmp/bin/*
PATH=/tmp/bin:$PATH WP_DIR=/tmp/wordpress bash plugins/tests/integration.sh
```

Resultat: **267 PASS / 0 FAIL**, exit 0, og smoke **1 568 PASS / 0 FAIL**. De nye sager der nu er dækket mod rigtig database og WordPress:

Område	Check	Målt
REST/kø	scheduled_at er site-wall-clock med timezone_string=Europe/Copenhagen	2026-12-25 10:00 gemmes som 2026-12-25 10:00:00
REST/kø	umulig dato 2026-02-31 10:00	400, ikke normaliseret til marts
REST/kø	force kommer som strengen "false"	422 mod bandet ord; "true" giver 201
REST/drafts	ukendt draft-id	404
REST/kø	status=queued på et fejlet element	200, attempts=0, error=null, remind=0; ukendt id 404
Kvalitetsgate	query-keys på en indlægs-URL	?utm_source er ikke arkiv; ?paged=2 er; ?p=1 er et enkelt indlæg
Uninstall	_sgap_evergreen + _sgap_ci_check på en rigtig post og sgap_coach_learn i cron	begge dele væk efter uninstall; 0 rækker og 0 events

To fejl blev fundet undervejs:

1. **Testens arkiv-forventning var forkert, ikke koden.** `home_url('/?utm_source=ci')` er forsiden og dermed et arkiv; det er først et enkeltindlæg, når queryen hænger på en indlægs-permalink.

Testen bruger nu `/ci-brass-goggles-guide/?utm_source=ci`.

2. **admin-tools.php havde et forældet ability-tal.** Den krævede 41, men 7.79.0 registrerer 43; testen er opdateret til 43. Tallet skal hæves i samme commit som en ny ability, ellers fanger integrationen det.

PHP 8.5 giver deprecation-linjer fra wp-cli's egne React- og php-cli-tools-filer. De er ikke pluginfejl; wp-shimmen kører med `-d error_reporting=8191` for at holde loggen læsbar. `debug.log`-kontrollen er stadig slået til og fandt ingen plugin-notices.